

VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF SCIENCE
FACULTY OF INFORMATION TECHNOLOGY



DEPARTMENT OF COMPUTER NETWORKS AND TELECOMMUNICATION

REPORT OF PROJECT

Project: Video Streaming System using RTSP/RTP protocols

Lecture:	Lê Hà Minh	
Leader:	Khang Phuc Nguyen	24120068
Members:	Nghia Trong Hoang	24120103
	Trong Minh Pham	24120151

Ho Chi Minh City, 12/2025

Table of Contents

1	Cơ sở lý thuyết	2
1.1	Mô hình kiến trúc và Giao diện Socket	2
1.2	Các giao thức tầng giao vận	2
1.3	Giao thức điều khiển RTSP	3
1.4	Giao thức vận chuyển RTP và Cấu trúc gói tin	4
1.5	Kỹ thuật Video Streaming và Xử lý bộ đệm	4
1.5.1	Định dạng MJPEG và Phân mảnh gói tin (Fragmentation)	4
1.5.2	Cơ chế Bộ đệm (Buffering) và Kiểm soát Jitter	4
1.6	Các mẫu thiết kế phần mềm (Design Patterns)	5
1.6.1	Mẫu thiết kế Builder (Builder Pattern)	5
1.6.2	Mẫu thiết kế Singleton (Singleton Pattern)	5
1.6.3	Mẫu thiết kế Facade (Facade Pattern)	6
2	Implementation	7
2.1	Tổ chức Codebase	7
2.2	Server Implementation	7
2.2.1	Cơ chế đa luồng (Multi-threading)	7
2.2.2	Xử lý yêu cầu RTSP (State Machine)	8
2.3	Client Implementation	9
2.3.1	Giao diện tương tác (ClientUI)	9
2.3.2	Gửi yêu cầu RTSP	10
2.4	Data Protocol (RTP)	10
2.4.1	Đóng gói gói tin (Packetization)	10



Giới thiệu

Với sự phát triển của công nghệ số và nhu cầu giải trí của con người tăng cao, đặc biệt khi cuộc cách mạng 4.0, sự bùng nổ của Internet thì vai trò của Mạng Máy Tính trở nên ngày càng quan trọng và có nhiều ứng dụng thực tiễn chẳng hạn là như là nhắn tin trực tuyến, làm việc trực tuyến và đặc biệt là trao đổi các thông tin, dữ liệu thông quan mạng. Đặc biệt trong số đó thì việc truyền các Video hoặc Ảnh thông qua được sử dụng ngày phổ biến trong các ứng dụng mạng xã hội như Facebook, TikTok và Youtube.

Trong dự án này, nhóm đã tìm hiểu và thực hiện hóa một hệ thống đơn giản cho phép hai ứng dụng mạng có thể giao tiếp với nhau thông qua giao tiếp RTSP/RTP để có thể phát một video trực tuyến. Dự án này được làm với mục đích học tập và hiểu rõ hơn về các giao thức cũng như là cách hoạt động của các ứng dụng trong thực tế, mặt khác, dự án này cũng chỉ là mô phỏng lại cách phát trực tiếp một video giữa các ứng dụng nên được xây dựng không quá phức tạp và tối ưu.

Toàn bộ mã nguồn của dự án được xây dựng bằng ngôn ngữ `C++ 17`, sử dụng `CMake` để có thể build trên cả Window và Linux, cũng như là sử dụng các thư viện `winsock2` và `POSIX sockets` để tạo các socket cho việc giao tiếp giữa các ứng dụng. Bên cạnh đó, nhóm cũng đã áp dụng các kiến thức về `Design Pattern` cho dự án với mục đích là mã nguồn của dự án có thể tái sử dụng, sạch sẽ và dễ đọc, dễ sửa lỗi cũng như là tối ưu trong quá trình hoạt động.

Để có thể có một giao diện cho ứng dụng của bên phía `client` thì nhóm đã sử dụng `Qt6` để tạo ra một giao diện đơn giản và dễ sử dụng cho người dùng. Toàn bộ quá trình, giao thức, phản hồi và yêu cầu của `server` và `client` được dựa trên toàn bộ yêu cầu được đính kèm trong file `Socket_2526` của thầy Lê Hà Minh [1]. Nhóm xin chân thành cảm ơn thầy đã hỗ trợ và cung cấp tài liệu để nhóm có thể hoàn thành đồ án này một cách tốt đẹp nhất.

Nhóm Trưởng

(Chữ ký và họ tên)

Khang

Nguyễn Phúc Khang

Ho Chi Minh City, 12/2025

1 Cơ sở lý thuyết

Hệ thống truyền phát video trong đề án được xây dựng dựa trên sự kết hợp chặt chẽ giữa kiến trúc mạng máy tính kinh điển và các kỹ thuật xử lý dữ liệu đa phương tiện thời gian thực, tuân thủ theo các yêu cầu của môn học [1]. Phần này sẽ trình bày chi tiết về các giao thức, kỹ thuật socket nâng cao và các mẫu thiết kế phần mềm được áp dụng.

1.1 Mô hình kiến trúc và Giao diện Socket

Hệ thống vận hành dựa trên mô hình **Client-Server**, trong đó trách nhiệm được phân chia rõ ràng giữa hai thực thể. **Server** đóng vai trò là bên cung cấp dịch vụ thụ động (*Passive Open*), chịu trách nhiệm lưu trữ tài nguyên video, quản lý các phiên làm việc và điều phối luồng dữ liệu. Ngược lại, **Client** đóng vai trò chủ động (*Active Open*), khởi tạo các yêu cầu kết nối và thực hiện tái tạo hình ảnh từ dữ liệu nhận được.

Để hiện thực hóa cơ chế giao tiếp này, dự án sử dụng **Socket API** như một giao diện lập trình cốt lõi [2]. Socket đóng vai trò là điểm cuối trong liên kết hai chiều giữa hai chương trình chạy trên mạng. Tùy thuộc vào yêu cầu về độ tin cậy hay tốc độ của từng loại dữ liệu, hệ thống sẽ khởi tạo loại socket phù hợp: **Stream Socket** (sử dụng TCP) cho kênh điều khiển nhằm đảm bảo tính chính xác tuyệt đối của lệnh, hoặc **Datagram Socket** (sử dụng UDP) cho kênh dữ liệu để tối ưu hóa độ trễ thời gian thực. Bên cạnh đó, các tùy chọn nâng cao như `SO_REUSEADDR` cũng được áp dụng để quản lý trạng thái cổng kết nối hiệu quả, tránh các lỗi khóa tài nguyên thường gặp trong lập trình mạng UNIX [2].

1.2 Các giao thức tầng giao vận

Việc lựa chọn giao thức tầng giao vận là quyết định kỹ thuật quan trọng nhất ảnh hưởng đến hiệu năng của ứng dụng streaming. Thay vì chỉ sử dụng một giao thức đơn lẻ, hệ thống áp dụng cơ chế lai (Hybrid) để tận dụng ưu điểm của cả TCP và UDP theo kiến trúc nền tảng về mạng máy tính [3]. Sự khác biệt và phạm vi áp dụng của hai giao thức này trong dự án được tóm tắt tại Bảng 1.

Table 1: So sánh và phạm vi áp dụng của TCP và UDP trong hệ thống

Giao thức	Đặc điểm kỹ thuật	Áp dụng trong dự án
TCP	Hướng kết nối, đảm bảo tin cậy, có kiểm soát luồng và tắc nghẽn, độ trễ cao hơn do cơ chế truyền lại (retransmission).	Kênh RTSP: Truyền tải các lệnh điều khiển quan trọng như SETUP, PLAY.
UDP	Phi kết nối, không đảm bảo tin cậy, header nhỏ (8 byte), độ trễ thấp, hỗ trợ truyền thời gian thực.	Kênh RTP: Vận chuyển dữ liệu video (JPEG). Chấp nhận mất gói để giữ độ trễ thấp.

1.3 Giao thức điều khiển RTSP

Real-Time Streaming Protocol (RTSP), được định nghĩa trong RFC 2326 [4], hoạt động như một "bộ điều khiển từ xa" qua mạng cho các máy chủ đa phương tiện. Điểm khác biệt cốt lõi của RTSP so với HTTP là tính chất *lưu trạng thái* (stateful). Server duy trì một phiên làm việc thông qua mã định danh **Session ID**, cho phép Server theo dõi trạng thái hiện tại của Client để phản hồi hợp lý.

Quá trình chuyển đổi trạng thái của hệ thống tuân theo quy trình nghiêm ngặt: từ trạng thái **INIT** (khởi tạo), chuyển sang **READY** (sẵn sàng) sau khi thiết lập tài nguyên thành công, và cuối cùng là **PLAYING** (đang phát) khi luồng dữ liệu bắt đầu được truyền tải. Các phương thức điều khiển chính được mô tả trong Bảng 2.

Table 2: Các phương thức RTSP và chức năng tương ứng

Phương thức	Mô tả chức năng
SETUP	Khởi tạo phiên làm việc. Client gửi cổng nhận RTP (<i>client_port</i>), Server phản hồi với <i>Session ID</i> và cổng truyền của Server.
PLAY	Yêu cầu Server bắt đầu truyền dữ liệu RTP. Server chuyển trạng thái sang PLAYING.
PAUSE	Tạm dừng luồng dữ liệu. Server ngừng gửi gói tin nhưng vẫn giữ nguyên phiên làm việc và trạng thái kết nối.
TEARDOWN	Kết thúc phiên làm việc. Server đóng kết nối, giải phóng bộ đệm và thu hồi Session ID.

1.4 Giao thức vận chuyển RTP và Cấu trúc gói tin

Trong khi RTSP chịu trách nhiệm điều khiển, **Real-time Transport Protocol (RTP)** theo tiêu chuẩn RFC 3550 [5] đảm nhiệm vai trò vận chuyển dữ liệu video thực tế. Do đặc thù của giao thức UDP là không đảm bảo thứ tự, RTP bổ sung một phần tiêu đề (Header) dài 12 byte vào trước mỗi gói dữ liệu để cung cấp thông tin tái đồng bộ.

Ba trường thông tin quan trọng nhất trong RTP Header bao gồm: **Sequence Number** (16 bit) giúp bên nhận phát hiện mất gói và sắp xếp lại các gói tin bị đảo lộn; **Timestamp** (32 bit) đóng vai trò nhận thời gian để đồng bộ hóa tốc độ hiển thị khung hình, đảm bảo video không bị tua nhanh hoặc chậm bất thường; và **Marker Bit** (1 bit) dùng để đánh dấu gói tin cuối cùng của một khung hình, hỗ trợ quá trình tái lắp ráp dữ liệu phân mảnh.

1.5 Kỹ thuật Video Streaming và Xử lý bộ đệm

Để đảm bảo khả năng truyền tải video mượt mà trên môi trường mạng biến động, hệ thống áp dụng các kỹ thuật xử lý dữ liệu nâng cao liên quan đến định dạng nén, phân mảnh và cơ chế đệm.

1.5.1 Định dạng MJPEG và Phân mảnh gói tin (Fragmentation)

Dự án sử dụng chuẩn nén **MJPEG (Motion JPEG)**, nơi mỗi khung hình video là một bức ảnh JPEG độc lập sử dụng kỹ thuật nén nội khung (intra-frame compression) [6]. Phương pháp này tuân thủ RFC 2435 [7] và mang lại khả năng chịu lỗi cao, vì việc mất một gói tin chỉ ảnh hưởng cục bộ đến khung hình hiện tại mà không gây lỗi dây chuyền cho các khung hình kế tiếp.

Tuy nhiên, kích thước của một khung hình video (đặc biệt ở độ phân giải HD) thường vượt quá giới hạn **MTU (Maximum Transmission Unit)** của mạng Ethernet (thông thường là 1500 byte). Để giải quyết vấn đề này, dự án áp dụng kỹ thuật **Phân mảnh cấp ứng dụng** (Application-Level Fragmentation). Một khung hình lớn sẽ được chia nhỏ thành nhiều gói RTP có kích thước tối đa 1400 byte. Hệ thống sử dụng mẫu thiết kế **Strategy Pattern** [8] để linh hoạt chuyển đổi thuật toán xử lý giữa video SD (có thể không cần phân mảnh) và video HD (bắt buộc phân mảnh), đảm bảo tính tương thích và hiệu năng cao nhất.

1.5.2 Cơ chế Bộ đệm (Buffering) và Kiểm soát Jitter

Hiện tượng **Jitter** (sự biến thiên độ trễ gói tin) là nguyên nhân chính gây ra tình trạng giật hình trong streaming [3]. Để khắc phục, Client được thiết kế hoạt động theo mô hình **Producer-**

Consumer với một **Frame Buffer** trung gian. Luồng nhận mạng đóng vai trò Producer, liên tục tái lắp ráp các gói tin RTP và đẩy khung hình hoàn chỉnh vào hàng đợi. Luồng hiển thị giao diện đóng vai trò Consumer, lấy khung hình ra để hiển thị theo định kỳ.

Đặc biệt, kỹ thuật **Pre-buffering** (Tiền tải) được áp dụng nhằm yêu cầu Client phải tích lũy đủ một lượng khung hình nhất định (ngưỡng N frames) trước khi bắt đầu phát. Cơ chế này giúp tạo ra một vùng đệm an toàn, triệt tiêu các dao động tức thời của mạng và đảm bảo trải nghiệm xem video liên tục.

1.6 Các mẫu thiết kế phần mềm (Design Patterns)

Để đảm bảo mã nguồn có khả năng mở rộng, dễ bảo trì và tuân thủ các nguyên lý lập trình hướng đối tượng, dự án đã áp dụng một số mẫu thiết kế phần mềm kinh điển được đề xuất bởi Gamma và cộng sự [8]. Việc áp dụng các mẫu này giúp giải quyết các vấn đề phổ biến trong việc khởi tạo đối tượng, quản lý trạng thái toàn cục và đơn giản hóa các giao diện phức tạp.

1.6.1 Mẫu thiết kế Builder (Builder Pattern)

Trong giao thức RTP, việc khởi tạo một gói tin (**RTPPacket**) đòi hỏi thiết lập rất nhiều tham số chi tiết cho phần tiêu đề (Header) như: phiên bản (Version), chỉ số đếm (Sequence Number), nhãn thời gian (Timestamp), bit đánh dấu (Marker), và loại tải (Payload Type). Việc sử dụng một hàm khởi tạo (Constructor) với quá nhiều tham số sẽ khiến mã nguồn khó đọc và dễ gây lỗi.

Để giải quyết vấn đề này, dự án áp dụng mẫu thiết kế **Builder Pattern**. Cụ thể, lớp **RTPPacketBuilder** (nằm trong thư mục `common/src/rtp`) được xây dựng để tách biệt quá trình khởi tạo phức tạp khỏi biểu diễn của đối tượng. Mẫu thiết kế này cho phép thiết lập từng thuộc tính của gói tin một cách tuần tự và rõ ràng thông qua chuỗi các phương thức (method chaining) trước khi gọi phương thức `build()` để tạo ra đối tượng gói tin hoàn chỉnh.

1.6.2 Mẫu thiết kế Singleton (Singleton Pattern)

Trong một hệ thống phân tán, có những thành phần cần đảm bảo tính duy nhất và có thể truy cập toàn cục từ bất kỳ đâu trong chương trình, điển hình là bộ ghi nhật ký (Logger) và bộ quản lý cấu hình (Configuration Manager).

Dự án áp dụng mẫu thiết kế **Singleton Pattern** cho các lớp **Logger** và **Config** (trong thư mục `common/src/utlis`). Mẫu thiết kế này đảm bảo rằng chỉ có duy nhất một thể hiện (instance)

của lớp **Logger** được tạo ra trong suốt vòng đời ứng dụng, giúp đồng bộ hóa việc ghi log từ nhiều luồng (thread) khác nhau vào một tệp tin hoặc màn hình console duy nhất, tránh tình trạng xung đột tài nguyên. Tương tự, **Config** đóng vai trò là kho chứa thông tin cấu hình trung tâm (như địa chỉ IP, cổng, kích thước bộ đệm) mà mọi thành phần khác đều có thể truy xuất.

1.6.3 Mẫu thiết kế Facade (Facade Pattern)

Lập trình mạng trên hệ điều hành thường yêu cầu tương tác với các API cấp thấp phức tạp của thư viện Berkeley Sockets (như `sockaddr_in`, `bind`, `listen`, `setsockopt`). Việc gọi trực tiếp các hàm C này rải rác trong mã nguồn sẽ làm giảm tính trừu tượng và khó chuyển đổi giữa các nền tảng (như Windows và Linux).

Để khắc phục, lớp **Socket** (trong thư mục `common/src/network`) được thiết kế như một **Facade** (mặt tiền). Lớp này cung cấp một giao diện lập trình hướng đối tượng đơn giản và thống nhất (với các phương thức như `send()`, `recv()`, `bind()`) để che giấu sự phức tạp của các lời gọi hệ thống bên dưới. Nhờ đó, các tầng cao hơn của ứng dụng (như RTSP Server hay RTP Sender) chỉ cần làm việc với đối tượng **Socket** mà không cần quan tâm đến chi tiết cài đặt của hệ điều hành.

Table 3: Tổng hợp các mẫu thiết kế được áp dụng trong cấu trúc mã nguồn

Mẫu thiết kế	Loại (Category)	Áp dụng thực tế trong dự án
Builder	Creational (Khởi tạo)	RTPPacketBuilder : Xây dựng gói tin RTP phức tạp từng bước một.
Singleton	Creational (Khởi tạo)	Logger , Config : Quản lý tài nguyên duy nhất (log file, cấu hình) toàn cục.
Facade	Structural (Cấu trúc)	Socket : Đóng gói và đơn giản hóa các thao tác với thư viện socket cấp thấp của hệ điều hành.
Strategy	Behavioral (Hành vi)	EncodingStrategy : Linh hoạt thay đổi thuật toán phân mảnh cho video SD và HD (đã đề cập ở phần trước).

2 Implementation

Phần này đi sâu vào chi tiết hiện thực của từng module trong hệ thống, bao gồm tổ chức mã nguồn, phân tích các lớp chính, trích dẫn các đoạn mã quan trọng và giải thích lý do lựa chọn giải pháp kỹ thuật cụ thể.

2.1 Tổ chức Codebase

Dự án được tổ chức theo cấu trúc thư mục phân tách rõ ràng giữa Client, Server và các thành phần dùng chung (Common) để đảm bảo tính module hóa và dễ bảo trì.

- `server/`: Chứa mã nguồn cho ứng dụng Server.
 - `src/network/RTSPServer.cpp`: Lớp chính để lắng nghe kết nối TCP.
 - `src/network/ServerWorker.cpp`: Lớp xử lý logic cho từng Client riêng biệt.
- `client/`: Chứa mã nguồn cho ứng dụng Client.
 - `src/ui/ClientUI.cpp`: Giao diện người dùng (Qt Framework).
 - `src/network/RTSPClient.cpp`: Xử lý gửi/nhận tín hiệu RTSP.
- `common/`: Chứa các thư viện dùng chung.
 - `src/network/Socket.cpp`: Wrapper cho API Socket của hệ điều hành.
 - `src/rtp/RTPPacket.cpp`: Logic đóng gói/giải gói dữ liệu RTP.

2.2 Server Implementation

2.2.1 Cơ chế đa luồng (Multi-threading)

Lý do thiết kế: Server phải có khả năng phục vụ nhiều Client xem video cùng lúc. Nếu chỉ sử dụng một luồng duy nhất (Single-thread), khi Server đang gửi dữ liệu cho Client A, Client B sẽ phải chờ đợi, gây ra độ trễ lớn.

Hiện thực: Trong `RTSPServer::run()`, chúng tôi sử dụng vòng lặp vô hạn để chấp nhận kết nối. Mỗi khi có kết nối mới, một `std::thread` mới được tạo ra để chạy `ServerWorker`.

```
1 void RTSPServer::run() {  
2     running = true;  
3     while (running) {
```

```
4 // Accept TCP connection from Client
5 std::unique_ptr<Socket> clientSocket = listenSocket->accept();
6 if (clientSocket) {
7     int clientId = nextClientId++;
8     // Create Worker to manage this Client
9     auto worker = std::make_unique<ServerWorker>(clientId,
std::move(clientSocket));
10
11     // WHY: Create separate thread to avoid blocking main loop
12     std::thread workerThread([w = worker.get()]() { w->run(); });
13     workerThread.detach(); // Detach thread
14
15     // Save session to list
16     activeSessions.push_back({clientId, std::move(worker)});
17 }
18 }
19 }
```

Listing 1: Vòng lặp chấp nhận kết nối trong RTSPServer.cpp

2.2.2 Xử lý yêu cầu RTSP (State Machine)

Lý do thiết kế: Giao thức RTSP yêu cầu client và server phải đồng bộ về trạng thái (Ví dụ: Không thể gửi lệnh **PLAY** khi chưa **SETUP**). Sử dụng máy trạng thái (State Machine) giúp kiểm soát chặt chẽ luồng nghiệp vụ này.

Hiện thực: Trong `ServerWorker::handleSetup`, ta kiểm tra trạng thái hiện tại phải là **INIT** mới cho phép chuyển sang **READY**.

```
1 std::string ServerWorker::handleSetup(const RTSPMessage::Request& request) {
2     // WHY: Validate State to ensure correct workflow
3     if (state_ != State::INIT)
4         return RTSPMessage::buildResponse(400, "Invalid State", request.cseq);
5
6     // ... Logic to open video file and get RTP port ...
7
8     // Generate random Session ID
```

```
9     sessionId_ = generateSessionId();
10    state_ = State::READY; // State transition
11
12    return RTSPMessage::buildResponse(200, "OK", request.cseq,
13        {"Session", sessionId_}, {"Transport", "..."});
14 }
```

Listing 2: Kiểm tra trạng thái trong ServerWorker.cpp

2.3 Client Implementation

2.3.1 Giao diện tương tác (ClientUI)

Lý do thiết kế: Việc xử lý mạng (Network I/O) thường tốn thời gian và có thể block (treo) giao diện nếu chạy trên cùng một luồng UI. Tuy nhiên, với Qt Framework, ta có thể sử dụng cơ chế Signal/Slot bất đồng bộ để xử lý các sự kiện nút bấm một cách mượt mà.

Hiện thực: Khi người dùng bấm nút **Setup**, hàm **onSetupClicked** được gọi. Nó thực hiện gửi yêu cầu xuống tầng mạng và cập nhật giao diện dựa trên kết quả trả về.

```
1 void ClientUI::onSetupClicked() {
2     // Send SETUP command via RTSPClient object
3     bool success = rtspClient_>sendSetup(videoFile_, clientRTPPort_);
4
5     if (success) {
6         state_ = State::READY;
7         // WHY: Update button states (Enable/Disable)
8         // to prevent invalid user actions (e.g. clicking Play before Setup)
9         updateButtonStates();
10
11         // Enable timeline slider
12         timelineSlider_>setEnabled(true);
13     } else {
14         Logger::getInstance().log(LogLevel::ERROR, "SETUP Failed");
15     }
16 }
```

Listing 3: Xử lý sự kiện Setup trong ClientUI.cpp

2.3.2 Gửi yêu cầu RTSP

Trong `RTSPClient`, các lệnh được đóng gói thành chuỗi ký tự theo đúng chuẩn RFC 2326. Việc xây dựng chuỗi thủ công giúp ta kiểm soát chính xác từng byte gửi đi.

```
1 bool RTSPClient::sendSetup(const std::string& videoFile, int clientRTPPort) {
2     std::ostringstream req;
3     // Build Request Line
4     req << "SETUP " << videoFile << " RTSP/1.0\r\n";
5     // Add required Headers
6     req << "CSeq: " << ++cseq_ << "\r\n";
7     req << "Transport: RTP/AVP/UDP;unicast;client_port=" << clientRTPPort << "-"
8         << (clientRTPPort + 1) << "\r\n";
9     req << "\r\n"; // End of Headers
10
11     // Send via TCP Socket
12     std::string resp = sendRtspRequest(req.str());
13
14     // ... Parse response ...
15 }
```

Listing 4: Tạo bản tin SETUP trong `RTSPClient.cpp`

2.4 Data Protocol (RTP)

2.4.1 Đóng gói gói tin (Packetization)

Lý do thiết kế: Dữ liệu video MJPEG rất lớn, không thể gửi nguyên một khung hình trong một gói UDP (vì giới hạn MTU). Ta cần sử dụng RTP (Real-time Transport Protocol) để chia nhỏ và đánh dấu thứ tự, giúp Client có thể ghép lại chính xác.

Hiện thực: Lớp `RTPPacket` sử dụng các phép toán thao tác bit (Bitwise operations) để đóng gói các trường Header (Sequence Number, Timestamp, SSRC) vào mảng byte.

```
1 void RTPPacket::encode() {
2     // Byte 0: Version(2), Padding(1), Extension(1), CC(4)
3     // WHY: Use Bitwise OR (|) and Shift (<<) to pack multiple fields into 1 byte
4     header_[0] = (version_ << 6) | (padding_ << 5) | (extension_ << 4) | (cc_ & 0x0F);
5 }
```

```
6 // Byte 1: Marker(1), PayloadType(7)
7 header_[1] = (marker_ << 7) | (payloadType_ & 0x7F);
8
9 // Byte 2,3: Sequence Number (16 bit)
10 // Split into 2 bytes (Big-Endian / Network Byte Order)
11 header_[2] = (sequenceNumber_ >> 8) & 0xFF;
12 header_[3] = sequenceNumber_ & 0xFF;
13
14 // ... Similar for Timestamp and SSRC ...
15 }
```

Listing 5: Đóng gói RTP Header trong RTPPacket.cpp

Cách làm này đảm bảo gói tin RTP tạo ra tương thích hoàn toàn với các phần mềm chuẩn (như VLC, Wireshark), đồng thời tối ưu hóa băng thông vì không gửi dư thừa bất kỳ byte nào.



List of Figures



List of Tables

1	So sánh và phạm vi áp dụng của TCP và UDP trong hệ thống	3
2	Các phương thức RTSP và chức năng tương ứng	3
3	Tổng hợp các mẫu thiết kế được áp dụng trong cấu trúc mã nguồn	6

References

- [1] L. H. Minh, “Socket programming assignment: Rtp video streaming,” Course Material, 2024, document Socket_2526, Ho Chi Minh University of Science.
- [2] W. R. Stevens, B. Fenner, and A. M. Rudoff, *Unix Network Programming, Volume 1: The Sockets Networking API*, 3rd ed. Boston, MA: Addison-Wesley Professional, 2004.
- [3] J. F. Kurose and K. W. Ross, *Computer Networking: A Top-Down Approach*, 8th ed. Boston, MA: Pearson, 2021, global Edition.
- [4] H. Schulzrinne, A. Rao, and R. Lanphier, “Real Time Streaming Protocol (RTSP),” IETF, RFC 2326, Apr. 1998. [Online]. Available: <https://datatracker.ietf.org/doc/html/rfc2326>
- [5] H. Schulzrinne, S. L. Casner, R. Frederick, and V. Jacobson, “RTP: A Transport Protocol for Real-Time Applications,” IETF, RFC 3550, Jul. 2003. [Online]. Available: <https://datatracker.ietf.org/doc/html/rfc3550>
- [6] A. M. Tekalp, *Digital Video Processing*, 2nd ed. Upper Saddle River, NJ: Prentice Hall, 2015.
- [7] L. M. Berc, W. C. Fenner, R. Frederick, and S. McCanne, “RTP Payload Format for JPEG-compressed Video,” IETF, RFC 2435, Oct. 1998. [Online]. Available: <https://datatracker.ietf.org/doc/html/rfc2435>
- [8] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA: Addison-Wesley, 1994.